OPENVINO · DEVICE BENCHMARK

CPU vs GPU vs NPU

디바이스 구성 확인과 동일 모델 실행 속도 비교

OpenVINO로 내 시스템의 CPU·GPU·NPU를 확인하고, 같은 모델을 각 디바이스에서 실행해 속도를 직접 비교해봅니다.

CONTENTS

목차

1 **CPU·GPU·NPU 이해하기** 03 – 04

세 디바이스의 차이와 왜 속도를 비교해봐야 하는지

2 **디바이스 드라이버 준비** 05 – 08

GPU·NPU를 쓰기 위한 드라이버 설치와 하드웨어 확인

3 **OpenVINO로 디바이스 확인** 09 – 11

available_devices와 상세 정보를 코드로 확인하고 직접 실습

4 **실행 속도 비교하기** 12 – 16

비교용 모델 준비 → benchmark_app으로 CPU·GPU·NPU 측정

5 **결과 해석과 실전 팁** 17 – 20

Throughput·Latency 읽는 법, 파이썬 코드 비교, 실전 팁

CPU · GPU · NPU, 무엇이 다른가

디바이스	무엇에 강한가	비고
CPU	범용 연산 — 어떤 모델이든 실행 가능	별도 드라이버 없이 항상 사용 가능한 기본값
GPU	병렬 연산에 강해 이미지·대형 모델에 유리	Intel Compute Runtime 드라이버 필요
NPU	저전력 AI 추론에 특화된 전용 연산 장치	최신 Intel Core Ultra 등 일부 기종에만 탑재

왜 디바이스별 속도를 비교해야 하나

같은 AI 모델이라도 어떤 디바이스에서 실행하느냐에 따라 처리 속도와 전력 소비가 크게 달라집니다. 배포 전에 내 시스템에서 어떤 디바이스가 가장 유리한지 직접 측정해보는 것이 중요합니다.

응답 속도

실시간 서비스는 지연시간(Latency)이 짧은 디바이스가 유리합니다.

처리량

대량 데이터를 한 번에 처리한다면 처리량(Throughput)이 중요합니다.

전력·발열

노트북·엣지 기기라면 NPU가 훨씬 적은 전력으로 동작합니다.

가용성

GPU·NPU가 없는 환경에서도 CPU는 항상 대안이 됩니다.

사전 준비물

1 Python 가상환경

이전 가이드에서 만든 .venv 환경을 그대로 사용합니다.

2 OpenVINO 설치 완료

pip install -U openvino 가 되어 있어야 합니다.

3 (선택) GPU·NPU 드라이버

내장 그래픽이나 NPU를 쓰려면 별도 드라이버가 필요합니다.

4 비교할 모델

벤치마크에 쓸 모델 하나가 필요합니다 (12페이지에서 직접 만듭니다).

GPU 드라이버 설치

설치

bash (.venv)

```
$ sudo apt update  패키지 목록 최신화
$ sudo apt install -y intel-opencl-icd intel-level-zero-gpu level-zero  Intel GPU 연산 런타임 설치
$ sudo usermod -aG render $USER  현재 사용자를 render 그룹에 추가
```

NOTE 그룹 변경은 로그아웃 후 다시 로그인해야 적용됩니다. 정확한 패키지명은 설치 시점의 OpenVINO GPU 설정 문서를 함께 확인하는 것이 안전합니다.

NPU 드라이버 설치

설치

bash

```
# github.com/intel/linux-npu-driver 의 Releases 페이지에서
# 최신 .deb 파일 세 개(compiler / fw / level-zero)를 내려받은 뒤
$ sudo apt install ./intel-driver-compiler-npu_*.deb ./intel-fw-npu_*.deb ./intel-level-zero-npu_*.deb
NPU 드라이버 설치
```

NOTE NPU는 Intel Core Ultra 등 일부 최신 프로세서에만 탑재되어 있습니다. 없는 시스템이라면 이 단계는 건너뛰어도 됩니다.

OS 차원에서 먼저 확인하기

확인

bash

```
$ lscpu | grep "Model name"  CPU 모델명 확인
$ lspci | grep -i vga  내장·외장 그래픽 장치 확인
$ lspci | grep -i "neural\|npu"  NPU 장치가 보이는 시스템도 있음
```

NOTE OpenVINO를 거치기 전에 하드웨어 자체가 인식되는지 먼저 확인하는 단계입니다.

04 · OPENVINO 확인

디바이스 목록 확인하기

확인

check_devices.py

```
from openvino import Core


core = Core()
print(core.available_devices)

# 예: ['CPU'] 또는 ['CPU', 'GPU', 'NPU']
```

디바이스 상세 이름 확인하기

확인

check_devices.py

```
from openvino import Core


core = Core()
for device in core.available_devices:
    name = core.get_property(device, "FULL_DEVICE_NAME")
    print(f"{device}: {name}")
```

NOTE 예: CPU: 13th Gen Intel(R) Core(TM) i7-1360P 처럼 실제 제품명까지 확인할 수 있습니다.

직접 해보기

확인

TASK

check_devices.py를 실행해 내 시스템의 available_devices 목록을 확인하고, 각 디바이스의 FULL_DEVICE_NAME까지 출력해보세요.

TIP

GPU나 NPU가 목록에 안 보인다면 드라이버가 설치되지 않았거나, 그룹 권한 적용을 위한 재로그인·재부팅이 필요한 경우가 많습니다.

비교용 모델 준비하기

실습

bash (.venv)

```
$ pip install ultralytics  모델 내보내기용 라이브러리 설치
$ python -c "from ultralytics import YOLO; YOLO('yolo26n.pt').export(format='openvino')"  OpenVINO IR로 변환
```

NOTE 실행하면 yolo26n_openvino_model/ 폴더에 .xml/.bin 형태의 OpenVINO IR 모델이 생성됩니다. 이 파일 하나로 모든 디바이스를 동일하게 비교합니다.

benchmark_app이란

측정

항목	설명
`-m <경로>`	벤치마크할 모델(.xml) 파일 경로를 지정합니다.
`-d CPU / GPU / NPU`	실행할 디바이스를 지정합니다.
`-hint latency`	지연시간을 최소화하는 방향으로 실행합니다.
`-hint throughput`	처리량을 최대화하는 방향으로 실행합니다.
`-t 10`	벤치마크를 실행할 시간(초)을 지정합니다.

CPU에서 벤치마크 실행

측정

bash (.venv)

```
$ benchmark_app -m yolo26n_openvino_model/yolo26n.xml -d CPU -hint latency  CPU로 실행
```

NOTE 실행이 끝나면 Throughput(FPS)과 Latency(ms) 값이 요약되어 출력됩니다. 이 값을 기록해두세요.

GPU에서 벤치마크 실행

측정

bash (.venv)

```
$ benchmark_app -m yolo26n_openvino_model/yolo26n.xml -d GPU -hint latency  GPU로 실행
```

NOTE CPU 결과와 같은 모델·같은 옵션으로 디바이스만 바꿔서 실행하는 것이 핵심입니다.

NPU에서 벤치마크 실행

측정

bash (.venv)

```
$ benchmark_app -m yolo26n_openvino_model/yolo26n.xml -d NPU -hint latency  NPU로 실행
```

NOTE NPU가 없거나 드라이버가 인식되지 않으면 이 명령은 오류를 냅니다. 9페이지의 available_devices에 NPU가 있는지 먼저 확인하세요.

결과 읽는 법

개념

Throughput (처리량)

- 1초에 몇 장(FPS)을 처리할 수 있는지를 나타냅니다.
- 대량의 이미지·데이터를 한꺼번에 처리할 때 중요합니다.
- 숫자가 클수록 더 많이, 더 빨리 처리한다는 뜻입니다.

Latency (지연시간)

- 요청 하나를 처리하는 데 걸리는 시간(ms)입니다.
- 실시간 응답이 필요한 챗봇·카메라 앱에서 중요합니다.
- 숫자가 작을수록 사용자가 더 빠르게 결과를 받습니다.

파이썬 코드로 직접 시간 측정하기

실습

compare_devices.py

```python
import time
from openvino import Core

core = Core()
model = core.read_model("yolo26n_openvino_model/yolo26n.xml")

for device in core.available_devices:
    compiled = core.compile_model(model, device)
    start = time.time()
    for _ in range(50):
        compiled([input_data])  # 실제 입력 텐서로 교체
    elapsed = time.time() - start
    print(f"{device}: {elapsed:.3f}초 / 50회")
```

NOTE benchmark_app 없이도 내 코드 안에서 디바이스별 속도를 직접 비교할 수 있습니다. input_data는 모델에 맞는 실제 입력으로 바꿔야 합니다.

실전 팁

1 정밀도가 속도를 좌우합니다

FP32/FP16/INT8 중 어떤 정밀도인지에 따라 디바이스별 속도차가 크게 달라집니다. GPU·NPU는 FP16·INT8에서 특히 유리합니다.

2 목적에 맞게 hint를 고르세요

응답 속도가 중요하면 -hint latency, 대량 처리가 중요하면 -hint throughput을 사용하세요.

3 발열·배터리도 고려하세요

노트북이라면 NPU 사용 시 발열과 배터리 소모가 크게 줄어드는 경우가 많습니다.

4 여러 번 측정해 평균 내기

첫 실행은 컴파일 시간이 포함될 수 있으니, 두세 번 반복 측정해 평균을 보는 것이 정확합니다.

고맙습니다

숫자로 직접 확인해야 진짜 내 시스템에 맞는 디바이스를 고를 수 있습니다.

CPU만 있어도 전혀 문제 없습니다. GPU·NPU는 있으면 더 빨라지는 선택지일 뿐입니다.

Q & A